



GRUP 1

2026

**DOCUMENTACIÓ
TÈCNICA – ES
XUXEMONS**

Prepared By :

Grup 1

Prepared For :

Carlos.M, Marc.C, Eric.G

Index

Index.....	2
Explicación General del Docker Compose.....	3
Creación del Docker Compose.....	5
Frontend.....	6
Instalación de la imagen de Angular.....	7
Nombre del Contenedor.....	7
Volumen (Frontend).....	7
Puertos (Frontend).....	7
Comandos (Fronted).....	8
Red (Frontend).....	8
Depends_on (Frontend).....	8
Backend.....	9
Instalación de la imagen de MySQL.....	10
Nombre del Contenedor.....	11
Volumen (Backend).....	11
Puertos (Backend).....	12
Variables de Entorno (backend).....	12
Red (basedatos).....	13
Depends_on (Backend).....	13
Comandos (Backend).....	13
Base de Datos.....	15
Instalación de la imagen de MySQL.....	16
Nombre del Contenedor.....	16
Variables de Entorno (basedatos).....	16
Volumen (basedatos).....	17
Puertos (basedatos).....	17
Red (basedatos).....	17
Healthcheck.....	18
Volumen.....	18
Redes.....	19
Comando para ejecutar DockerCompose.....	19
Comprobación de Contenedores y Redes.....	20
Comprobación Contenedores y Redes.....	20
Comprobación Imágenes.....	20
Comprobación Volúmenes.....	21
Comprobación Redes PortNavigator.....	22

Explicación General del Docker Compose

En este apartado explicaremos cómo crear una docker-compose.yml para poder crear dentro del docker compose los servicios del (Backend y Frontend) con sus contenedores, volúmenes y sus redes.

```
1  name: xuxemonpro
2
3  services:
4    # Frontend (Angular)
5    frontend:
6      image: node:20-alpine
7      container_name: xuxemon-frontend
8      volumes:
9        - ./frontend:/app
10     ports:
11       - "4200:4200"
12     command: sh -c "cd /app && npm install && npm start -- --host 0.0.0.0"
13     networks:
14       - red
15     depends_on:
16       backend:
17         condition: service_started
18
19   # Backend (Laravel)
20   backend:
21     build:
22       context: ./backend
23       dockerfile: Dockerfile
24     container_name: xuxemon-backend
25     volumes:
26       - ./backend:/var/www/html
27     ports:
28       - "8000:80"
29     environment:
30       DB_CONNECTION: mysql
31       DB_HOST: basedatos
32       DB_PORT: 3306
33       DB_DATABASE: xuxemonsbd
34       DB_USERNAME: dev_user
35       DB_PASSWORD: 1234
36     networks:
37       - red
38     depends_on:
39       basedatos:
40         condition: service_healthy
41     command: >
42       sh -c "
43         php artisan config:clear &&
44         php artisan jwt:secret --force &&
45         php artisan migrate --force &&
46         php artisan db:seed --force &&
47         apache2-foreground
48       "
```

```
50 # Base de Datos (MySQL)
51 basedatos:
52   image: mysql:8.0
53   container_name: mysql-db
54   environment:
55     MYSQL_DATABASE: xuxemonsbd
56     MYSQL_USER: dev_user
57     MYSQL_PASSWORD: 1234
58     MYSQL_ROOT_PASSWORD: root_password
59   volumes:
60     - mysql_data:/var/lib/mysql
61   ports:
62     - "3306:3306"
63   networks:
64     - red
65   healthcheck:
66     test:
67       [
68         "CMD",
69         "mysqladmin",
70         "ping",
71         "-h",
72         "localhost",
73         "-u",
74         "root",
75         "-proot_password",
76       ]
77     interval: 10s
78     timeout: 5s
79     retries: 5
80
81   volumes:
82     mysql_data:
83
84   networks:
85     red:
86     driver: bridge
```

Creación del Docker Compose

Nuestro nombre del Dock Compose es el siguiente que vemos en la imagen:

```
name: xuxemonpro
```

Después vienen los servicios es donde definimos los contenedores que en este caso son el frontend, backend y Base de Datos que son los siguientes que vemos:

También es importante tener en cuenta las tabulaciones

```
services:
```

- frontend (Angular)
- backend (Laravel)
- basedatos (MySQL)



Frontend

En este apartado veremos el código en general de backend veremos cómo funciona cada paso en general de las backend , como vemos en la siguiente imagen:

```
# Frontend (Angular)
frontend:
  image: node:20-alpine
  container_name: xuxemon-frontend
  volumes:
    - ./frontend:/app
  ports:
    - "4200:4200"
  command: sh -c "cd /app && npm install && npm start -- --host 0.0.0.0"
  networks:
    - red
  depends_on:
    backend:
      condition: service_started
```



Instalación de la imagen de Angular

El Docker Compose en el Servicio de (frontend), primero descarga la imagen oficial de node:20-alpine que sería la siguiente imagen que vemos:

```
image: node:20-alpine
```

Nombre del Contenedor

Dentro del Servicio de frontend, después de la imagen tendremos el nombre del contenedor a la cual le llamaremos, como vemos en la siguiente imagen:

```
container_name: xuxemon-frontend
```

Volumen (Frontend)

En el frontend tenemos un volumen donde montamos en la carpeta local (./frontend) dentro del contenedor en (/app), esto sirve para que se refleje al instante dentro del contenedor sin necesidad de reconstruirlo el contenedor de nuevo, como vemos en la siguiente imagen:

```
volumes:  
  - ./frontend:/app
```

Puertos (Frontend)

En los puertos exponemos el puerto 4200 de nuestro contenedor 4200. La cual su función principal es hacer accesible los servicios interno de un contenedor a un host o redes externas como vemos en la siguiente imagen:

```
ports:  
  - "4200:4200"
```

Comandos (Fronted)

En el comando entra en la carpeta (app) para instalar las dependencias de angular, inicializando angular con un host local en este caso como vemos en la siguiente imagen:

```
command: sh -c "cd /app && npm install && npm start -- --host 0.0.0.0"
```

Red (Frontend)

El servicio de frontend lo conectamos a través de la red que es un bridge para conectarlo a otros servicios:

```
networks:  
  - red
```

Depends_on (Frontend)

En este apartado el Docker esperará a que el contenedor del backend haya arrancado. Para esperar a que un servicio como backend esté completamente operativo utilizamos el service_healthy, que esto requiere de un healthcheck que más adelante lo explicamos a detalle. Veremos en la siguiente imagen como vemos el depends_on:

```
depends_on:  
  backend:  
    condition: service_started
```

Backend

En este apartado veremos el código en general de backend veremos cómo funciona cada paso en general de las backend , como vemos en la siguiente imagen:

```
19      # Backend (Laravel)
20      backend:
21          build:
22              context: ./backend
23              dockerfile: Dockerfile
24          container_name: xuxemon-backend
25          volumes:
26              - ./backend:/var/www/html
27          ports:
28              - "8000:80"
29          environment:
30              DB_CONNECTION: mysql
31              DB_HOST: basedatos
32              DB_PORT: 3306
33              DB_DATABASE: xuxemonsbd
34              DB_USERNAME: dev_user
35              DB_PASSWORD: 1234
36          networks:
37              - red
38          depends_on:
39              basedatos:
40                  condition: service_healthy
41          command: >
42              sh -c "
43                  composer install --no-interaction --prefer-dist &&
44                  php artisan config:clear &&
45                  php artisan jwt:secret --force &&
46                  php artisan migrate --force &&
47                  php artisan db:seed --force &&
48                  apache2-foreground
49              "
50
```

Instalación de la imagen de MySQL

El Docker Compose en el Servicio de ([backend](#)), en vez de utilizar una imagen ya hecha la construimos a partir del Dockerfile con la imagen de apache que son los siguientes apartados:

```
build:
  context: ./backend
  dockerfile: Dockerfile
```

- Dependencias del sistema
- Extensiones PHP de laravel
- Instalación del Docker Compose
- Habilitamos el Mod_rewrite de Apache : activa el módulo de reescritura de URLs de Apache
- Configuración del Apache para Laravel
- El Workdir: es una instrucción dentro del Dockerfile que establece el directorio de trabajo actual para las instrucciones subsiguientes (RUN, CMD, ENTRYPOINT, COPY, ADD) y define la carpeta predeterminada al iniciar el contenedor.
- Exponemos el puerto 80

En el siguiente apartado vemos lo que incluye dentro del Dockerfile:

```
1  FROM php:8.3-apache
2
3  # Instalar dependencias del sistema
4  RUN apt-get update && apt-get install -y \
5      git \
6      curl \
7      libpng-dev \
8      libonig-dev \
9      libxml2-dev \
10     libzip-dev \
11     zip \
12     unzip \
13     && rm -rf /var/lib/apt/lists/*
14
15 # Instalar extensiones PHP para Laravel
16 RUN docker-php-ext-install \
17     pdo_mysql \
18     mbstring \
19     exif \
20     pcntl \
21     bcmath \
22     gd \
23     zip
```

```
25 # Instalar Composer
26 COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
27
28 # Habilitar mod_rewrite de Apache
29 RUN a2enmod rewrite
30
31 # Configurar Apache para Laravel
32 RUN echo '<VirtualHost *:80>\n\
33     DocumentRoot /var/www/html/public\n\
34     <Directory /var/www/html/public>\n\
35         AllowOverride All\n\
36         Require all granted\n\
37     </Directory>\n\
38     ErrorLog ${APACHE_LOG_DIR}/error.log\n\
39     CustomLog ${APACHE_LOG_DIR}/access.log combined\n\
40 </VirtualHost>' > /etc/apache2/sites-available/000-default.conf
41
42 WORKDIR /var/www/html
43
44 EXPOSE 80
```

Nombre del Contenedor

Dentro del Servicio de Backend, después de la imagen tendremos el nombre del contenedor a la cual le llamaremos, como vemos en la siguiente imagen:

```
container_name: xuxemon-backend
```

Volumen (Backend)

En el frontend tenemos un volumen donde montamos en la carpeta local (./backend) dentro del contenedor en (/var/www/html), esto sirve para que se refleje al instante dentro del contenedor sin necesidad de reconstruirlo el contenedor de nuevo, como vemos en la siguiente imagen:

```
volumes:
  - ./backend:/var/www/html
```

Puertos (Backend)

En los **puertos exponemos** el puerto **8000** de nuestro contenedor **80**. La cual su función principal es hacer accesible los servicios interno de un contenedor a un host o redes externas como vemos en la siguiente imagen:

```
ports:
  - "8000:80"
```

Variables de Entorno (backend)

En la Base de datos tenemos un apartado con variables de entorno para poder configurar la base de datos por primera vez y para poder entrar y ver la base de datos con sus tablas creadas, que cada variable de entorno lo que hace es los siguiente:

- **DB_CONNECTION:**
Tipo de Conexión (mysql)
- **DB_HOST:**
Dirección del servidor de la base de datos
- **DB_PORT:**
Puerto de la base de datos
- **DB_DATABASE:**
Nombre de la base de datos (xuxemonsbd)
- **DB_USERNAME:**
Usuario base de datos (xuxemonsbd)
- **DB_PASSWORD:**
Contraseña base de datos (xuxemonsbd)

```
environment:
  DB_CONNECTION: mysql
  DB_HOST: basedatos
  DB_PORT: 3306
  DB_DATABASE: xuxemonsbd
  DB_USERNAME: dev_user
  DB_PASSWORD: 1234
```

Red (basedatos)

El servicio de basedatos lo conectamos a través de la red que es un bridge para conectarlo a otros servicios:

```
networks:  
  - red
```

Depends_on (Backend)

En este apartado el Docker esperará a que el contenedor de basedatos esté completamente operativo antes de arrancar el backend. A diferencia del frontend que usa `service_started`, aquí se utiliza `service_healthy`, lo que significa que Docker no solo espera a que MySQL haya arrancado, sino que además exige que supere un healthcheck real, es decir, una prueba que confirma que la base de datos está lista para aceptar conexiones. Esto es imprescindible porque el backend necesita ejecutar `php artisan migrate` para crear las tablas, y si MySQL todavía se está inicializando en ese momento, la conexión fallaría y el contenedor se caería.

```
depends_on:  
  basedatos:  
    condition: service_healthy
```

Comandos (Backend)

En este apartado el Docker esperará a que el contenedor de basedatos esté completamente operativo antes de arrancar el backend. A diferencia del

El Docker Compose en el servicio de backend, una vez que el contenedor está en pie, ejecuta una serie de comandos en orden estricto que preparan Laravel antes de arrancar Apache:

- `composer install --no-interaction --prefer-dist` : instala todas las dependencias PHP del proyecto leyendo el archivo `composer.json`, equivalente a lo que sería un `npm install` en Node.

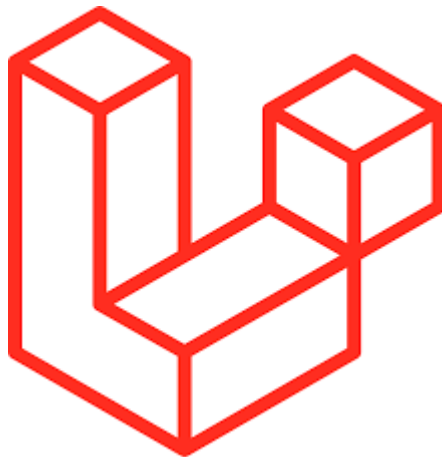
- `php artisan config:clear` : limpia la caché de configuración para que Laravel lea correctamente las variables de entorno que le hemos pasado en el apartado `environment`, como el `host` o las credenciales de la base de datos.
- `php artisan jwt:secret --force` : genera una clave secreta aleatoria que se usará para firmar y verificar los tokens JWT, que son los que gestionan la autenticación de los usuarios.
- `php artisan migrate --force` : crea todas las tablas de la base de datos a partir de los archivos de migración de Laravel. El `--force` es necesario porque estamos en un entorno que Laravel detecta como producción y sin él pediría confirmación manual.
- `php artisan db:seed --force` : rellena las tablas recién creadas con datos iniciales definidos en los seeders, como usuarios por defecto o datos de prueba.
- `apache2-foreground` : arranca Apache en primer plano. Esto es imprescindible porque Docker mata un contenedor si su proceso principal termina, por lo que Apache debe mantenerse activo en primer plano para que el contenedor siga vivo.

```
command: >
  sh -c "
    composer install --no-interaction --prefer-dist &&
    php artisan config:clear &&
    php artisan jwt:secret --force &&
    php artisan migrate --force &&
    php artisan db:seed --force &&
    apache2-foreground
  "
```

Base de Datos

En este apartado veremos el código en general de base de datos veremos cómo funciona cada paso en general de las bases de datos, como vemos en la siguiente imagen:

```
51 # Base de Datos (MySQL)
52 basedatos:
53   image: mysql:8.0
54   container_name: mysql-db
55   environment:
56     MYSQL_DATABASE: xuxemonsbd
57     MYSQL_USER: dev_user
58     MYSQL_PASSWORD: 1234
59     MYSQL_ROOT_PASSWORD: root_password
60   volumes:
61     - mysql_data:/var/lib/mysql
62   ports:
63     - "3306:3306"
64   networks:
65     - red
66   healthcheck:
67     test:
68       [
69         "CMD",
70         "mysqladmin",
71         "ping",
72         "-h",
73         "localhost",
74         "-u",
75         "root",
76         "-proot_password",
77       ]
78     interval: 10s
79     timeout: 5s
80     retries: 5
```



Instalación de la imagen de MySQL

El Docker Compose en el Servicio de ([basedatos](#)), primero descarga la imagen oficial de MySQL 8.0 que sería la siguiente imagen que vemos:

```
image: mysql:8.0
```

Nombre del Contenedor

Dentro del Servicio de base datos después de la imagen tendremos el nombre del contenedor a la cual le llamaremos, como vemos en la siguiente imagen:

```
container_name: mysql-db
```

Variables de Entorno (basedatos)

En la Base de datos tenemos un apartado con variables de entorno para poder configurar la base de datos por primera vez y para poder entrar y ver la base de datos con sus tablas creadas, que cada variable de entorno lo que hace es los siguiente:

- **MYSQL_DATABASE:**
Nombre de la Base de Datos
- **MYSQL_USER:**
Usuario para iniciar en la base de datos
- **MYSQL_PASSWORD:**
Contraseña del Usuario
- **MYSQL_ROOT_PASSWORD:**
Contraseña del Root

```
environment:  
  MYSQL_DATABASE: xuxemonsbd  
  MYSQL_USER: dev_user  
  MYSQL_PASSWORD: 1234  
  MYSQL_ROOT_PASSWORD: root_password
```

Volumen (basedatos)

En el volumen de Docker mysql_data en la carpeta de MySQL guarda sus datos físicamente. Así cuando hagamos un docker compose down los datos no se borran, que el volumen es el siguiente que vemos en la imagen:

```
volumes:  
  - mysql_data:/var/lib/mysql
```

Puertos (basedatos)

En los [puertos exponemos](#) el puerto 3306 de nuestro contenedor 3306. La cual su función principal es hacer accesible los servicios interno de un contenedor a un host o redes externas como vemos en la siguiente imagen:

```
ports:  
  - "3306:3306"
```

Red (basedatos)

El servicio de basedatos lo conectamos a través de la red que es un bridge para conectarlo a otros servicios:

```
networks:  
  - red
```

Healthcheck

El **Healthcheck** la función que tiene en el apartado de la base de datos es verificar que el servicio de la Base de datos, tiene 5 intentos de 10 segundos cada intento, si falla 5 veces seguidas, pasa a **unhealthy** y no se iniciará MySQL.

Además el **Backend** tiene la (**condition: service_healthy**), con este apartado no arrancará hasta que el **backend** se conecta a **MySQL**.

```
healthcheck:
  test:
    [
      "CMD",
      "mysqladmin",
      "ping",
      "-h",
      "localhost",
      "-u",
      "root",
      "-proot_password",
    ]
  interval: 10s
  timeout: 5s
  retries: 5
```

Volumen

El Volumen guarda los registros en general y las migraciones que incluyamos dentro de la base de datos.

En el caso de que la Base de datos se destruya, reinicie o se corrompa, así los datos que estén introducidos se guardan en la siguiente ruta que vemos en la imagen, para que así se conserven los datos si pasa algún problema.

```
volumes:
  | mysql_data:
```

Redes

La Red, lo que nos permite es que se conecte ambos servicios que son el frontend y la base de datos una red virtual privada, lo cual esta red es un **bridge** que sirve para conectar a través de un puente ambos servicios.

```
networks:  
  red:  
    driver: bridge
```

Comando para ejecutar DockerCompose

En el proyecto de xuxemons tendremos que ejecutar por primera vez el siguiente comando para que se creen las imágenes, contenedores, volúmenes, redes, que es el siguiente que vemos en la siguiente imagen:

```
$ docker compose up -d
```

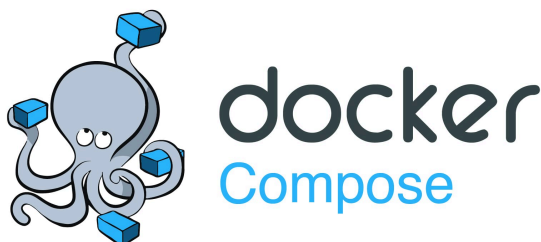
Con el siguiente comando eliminaremos todo los servicios con sus respectivos datos de la base de datos que es el siguiente comando que vemos:

(el -v lo que hace es borrar los datos persistentes y los volúmenes asociados)

```
$ docker compose down -v
```

En el siguiente comando podremos utilizar para poderlo reconstruir todo

```
$ docker compose up -d --build
```



Comprobación de Contenedores y Redes

En el Docker Desktop comprobaremos que se hayan creado las imágenes, contenedores, volúmenes, redes etc... como vemos en las siguientes imágenes:

Comprobación Contenedores y Redes

En la imagen como podemos observar, podemos ver que se ha creado los contenedores con sus respectivas redes cada una:

Containers [Give feedback](#)

Container CPU usage 284.94% / 800% (8 CPUs available) Container memory usage 714.4MB / 7.52GB [Show charts](#)

Search Only show running containers

	Name	Container ID	Image	Port(s)	CPU (...)	Last started	Actions
☐	tender_wing	db0beacea21b	php:latest		0%	2 months ago	
☒	xuxemonpro	-	-	-	229.6%	1 minute ago	
☐	xuxemon-frontend	bb138e035f80	node:20-alpine	4200:4200	228.97%	1 minute ago	
☐	xuxemon-backend	e26dd5590ecc	xuxemonpro-backend	8000:80	0%	1 minute ago	
☐	mysql-db	024c01f8f3b5	mysql:8.0	3306:3306	0.63%	1 minute ago	

Comprobación Imágenes

En la imagen como podemos observar, podemos ver que se ha creado las imágenes con sus respectivas redes cada una:

Images [Give feedback](#)

Local My Hub

2.17 GB / 5.39 GB in use 5 images Last refresh: 21 hours ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☒	xuxemonpro-backend	latest	d2104dab66f5	15 days ago	797.18 MB	
☒	php	latest	7c99b9bd905e	2 months ago	795.64 MB	
☒	mysql	8.0	99d774bf02a4	3 months ago	1.08 GB	
☒	node	20-alpine	09e2b3d97260	3 months ago	192.04 MB	
☐	portnavigator/port-navigator	1.1.0	1c3b1d792e15	3 years ago	235.45 MB	

Comprobación Volúmenes

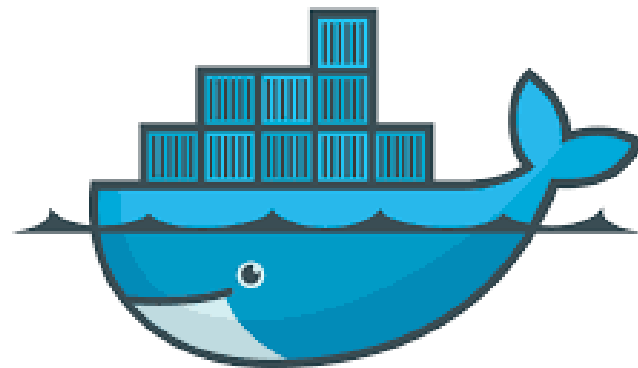
En la imagen como podemos observar, podemos ver que se ha creado los volúmenes con sus respectivas redes cada una:

Volumes [Give feedback](#)

Search

Create

	Name	Created	Size	Actions
☐	xuxemonpro_mysql_data	15 days ago	219 MB	



docker



Comprobación Redes PortNavigator

En la extensión del PortNavigator podemos comprobar que los contenedores están conectados en la misma red a través de un puente que hay en la red de Xuxemons.

The screenshot displays the 'Network: xuxemonpro_red' interface in PortNavigator. At the top, it shows the network configuration: Driver: bridge, Gateway: 172.18.0.1, and Subnet: 172.18.0.0/16. Below this, there are three buttons: 'Connected Containers' (showing 3), 'Hide Containers', and 'Add Container'. The interface lists three containers connected to the network:

- Container: xuxemon-frontend**
 - ContainerID: bb138e035f80508b9a101...
 - Image: node:20-alpine
 - Activity: Up About a minute
 - Type: tcp
 - IPv4Address: 172.18.0.4/16
 - Published Ports: 4200:4200, 4200:4200
 - Private Ports: null
 - Buttons: Disconnect, Connect to Networks, Edit Ports
- Container: xuxemon-backend**
 - ContainerID: e26dd5590ecc3782756ffb...
 - Image: xuxemonpro-backend
 - Activity: Up About a minute
 - Type: tcp
 - IPv4Address: 172.18.0.3/16
 - Published Ports: 8000:80, 8000:80
 - Private Ports: null
 - Buttons: Disconnect, Connect to Networks, Edit Ports
- Container: mysql-db**
 - ContainerID: 024c01f8f3b5b128f2ca6d...
 - Image: mysql:8.0
 - Activity: Up About a minute (healthy)
 - Type: tcp
 - IPv4Address: 172.18.0.2/16
 - Published Ports: 3306:3306, 3306:3306
 - Private Ports: null
 - Buttons: Disconnect, Connect to Networks, Edit Ports